- *Complexity*—The application becomes too complex to add any new features. Even the simplest change takes a huge amount of effort, and it gets harder to test the application properly, causing ongoing worry: What will fail this time?
- *Inflexibility*—The application isn't adaptive. With every big jump in the number of users, the application has to be rewritten from scratch. Each rewrite takes a long time and is complex. You have more users than you can handle, and you're split between keeping the existing application running and rewriting it to support more users.

Let's say you have been building applications for a while and choose to build this one the way you have in the past, taking the traditional approach: using low-level threads, locks, and RPCs; blocking I/O; and—first on the menu, in the next section—mutating state in a database.

1.4 Traditional scaling

You start on one server. You set out to build the first version of the chat application and come up with a data-model design, shown in figure 1.3. For now, you keep these objects in memory.